



MODULE 13

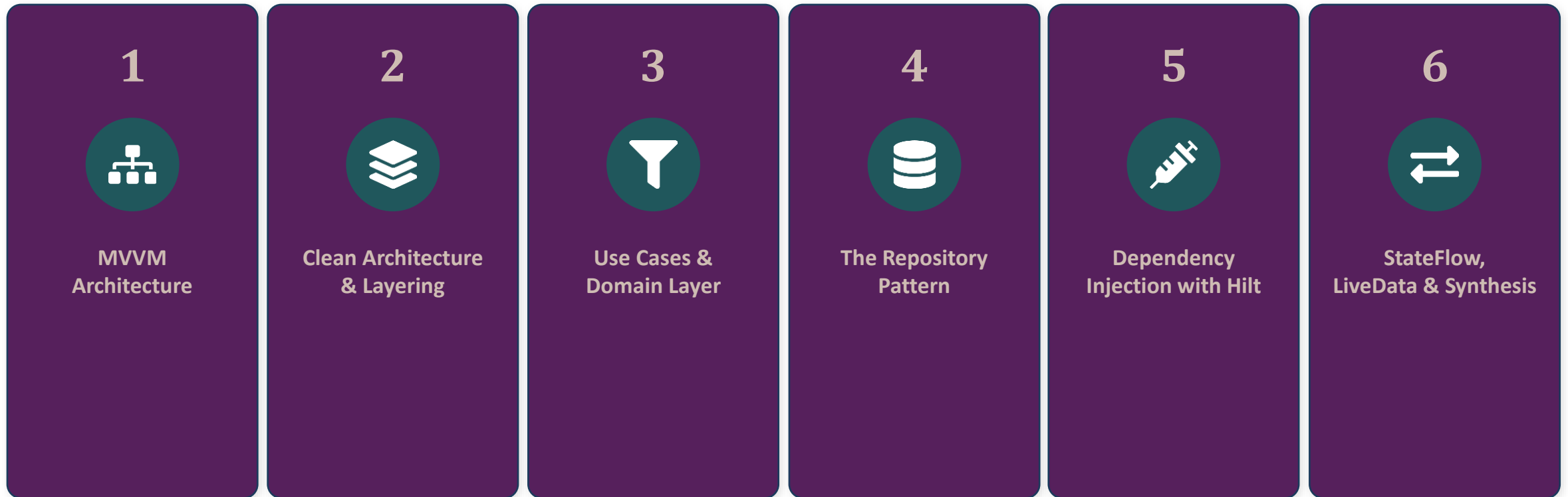
Android Architecture Patterns

Enterprise Mobile Application Development Bootcamp | AI-Integrated Edition

Phase 03 | Android Development with Kotlin

Module 13 Roadmap

Six blocks. By the end: a refactored, testable, Hilt-injected PNC Mobile Android feature.



MVVM Architecture

- The Massive Activity Problem
- Extracting the ViewModel
- the “Zero Android Framework Imports” Test



The Massive Activity Problem

```
class AccountsActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        lifecycleScope.launch {  
            val response = httpClient.get("https://api.pncmobile.com/accounts")  
            val accounts = Json.decodeFromString<List<Account>>(response.body())  
            for (account in accounts) {  
                if (account.balance < 0) showOverdraftWarning(account)    // business rule  
            }  
            recyclerView.adapter = AccountAdapter(accounts)                // UI update  
        }  
    }  
    // ...plus click listeners, error dialogs, formatting, navigation...  
}
```



ViewModels in Compose

- In many frameworks a ViewModel is simply any object that you use together with a view
- In Jetpack Compose, ViewModel is a class with very specific functionality that you inherit
- The ViewModel's job is to expose state and business logic to the UI
 - It caches its state and persists it through configuration changes such as screen rotation
 - This means that the UI doesn't have to fetch data again when navigating between activities, rotating the screen, or changing posture (such as opening/closing a foldable)
- ViewModels are used in conjunction with StateFlow



ViewModel Dependencies

- The Lifecycle Viewmodel Compose and the Lifecycle Runtime Compose artifacts are needed for ViewModel functionality
- These should be added to your app's build.gradle file

Code Bite

```
implementation  
"androidx.lifecycle:lifecycle-viewmodel-  
compose:2.8.7"
```

```
implementation  
"androidx.lifecycle:lifecycle-runtime-  
compose:2.8.7"
```



Accessing the ViewModel

- The `viewModel()` function finds the closest `ViewModelStoreOwner` and fetches the active `ViewModel` instance
 - This ensures state survives configuration and posture changes
 - (`ViewModelStoreOwner` is your host Activity, Fragment, or Navigation BackStack entry)
- It uses a fetch-or-create logic to ensure that you will always receive an instance of the `ViewModel`

Code Bite

```
@Composable
fun AccountProfileRoute(
    viewModel: AccountProfileViewModel
        = viewModel()
) {

}
```



StateFlow

- **StateFlow** is the standard and recommended way to expose business logic and screen state from a ViewModel to the UI
 - It is observable
- State changes should only be made through the ViewModel
 - Keep a private MutableStateFlow
 - Expose an immutable view of the StateFlow as a public property for the UI to observe

Code Bite

```
private val _uiState = MutableStateFlow(  
    AccountProfileState  
)  
  
public uiState:  
    StateFlow<AccountProfileState> =  
        _uiState.asStateFlow()
```



MVVM: Extracting the ViewModel

```
class AccountsViewModel(private val repository: AccountsRepository) : ViewModel() {  
    private val _accounts = MutableStateFlow<List<Account>>(emptyList())  
    val accounts: StateFlow<List<Account>> = _accounts  
  
    fun loadAccounts() {  
        viewModelScope.launch {  
            _accounts.value = repository.fetchAccounts()  
        }  
    }  
  
    val overdraftAccounts: List<Account>  
        get() = _accounts.value.filter { it.balance < 0 }  
}
```



remember

- **remember** stores state directly in the composition tree memory
- It acts as a local cache that is retained only as long as the composable remains active in the UI hierarchy
- **Best for:** very temporary, short-lived, or local visual properties that are easy to recreate if lost
- **Examples:** an animation state, a Boolean flag tracking whether a dropdown menu or dialog is open, or an intermediate expanded/collapsed card state



rememberSaveable

- **rememberSaveable** works similarly to **remember**, but it hooks into Android's native savedInstanceState mechanism
- This allows it to serialize your state into a Bundle, preserving it even if the screen rotates or the operating system terminates the app process in the background
- **Limitations:** The data must be standard primitives (like strings, integers, booleans) or explicitly implement Parcelable
 - It is restricted by small Bundle size limits and should not handle massive objects
- **Best For:** Small items of user input or UI state that a user expects to persist during orientation changes or app interruptions
- **Examples:** Current text in an unsubmitted form field, current scroll position of a list, or the currently active navigation tab



ViewModel

- A **ViewModel** is an architectural component tied to the lifecycle of a screen (like an Activity or Navigation graph destination) rather than a single Composable
- It acts as the "brain of the UI," managing heavy business logic and keeping data intact across configuration changes like screen rotations
- **Note on Process Death:** A standard ViewModel *does not* automatically survive process death
 - You must inject a SavedStateHandle into it to backup crucial pieces of state
- **Best For:** Complex data structures, handling network requests, querying a database, or sharing state between multiple composable components on a screen
 - ViewModels also move testing logic out of the UI layer, enabling fast local JVM unit tests
- **Examples:** A user profile object fetched from an API, search results from a database repository, or managing complex screen state via state management architectures (like a MutableStateFlow of your UI state)



remember vs. rememberSaveable vs. ViewModel

Feature/Behavior	remember	rememberSaveable	ViewModel
Survives recomposition?	Yes	Yes	Yes
Survives screen rotation?	No	Yes	Yes
Survives process death	No	Yes	No (Yes w/ SavedStateHandle)
Data type support	Any object type	Only Bundle/Parcelable	Any object type
Primary scope	Local/Ephemeral UI	Small, persistent UI state	Screen-level/Business logic



Block 1 - MVVM Architecture

Skill Check ✓

- Name three distinct responsibilities crammed into the Massive Activity example.
- What's the literal, checkable test for whether business logic has leaked into an Activity?
- In the MVVM version, what's left for the Activity to actually do?



Clean Architecture & Layering

- The Dependency Rule
- Presentation, Domain & Data Layers
- Mapping Layers onto an Android Project



Clean Architecture

- Clean Architecture is a software design philosophy introduced by Robert C. Martin ("Uncle Bob")
- It organizes code into concentric layers to keep core business logic completely independent of external concerns like databases, user interfaces, and frameworks
- Its principles include:
- **Inward Flow:** Source code dependencies can only point inward
- **Inner Isolation:** The innermost layers know nothing about the outer layers. They do not mention functions, classes, or variables from the outside
- **Outer Adaptability:** Outer layers can change frequently (like swapping a SQL database for a NoSQL store or changing a web framework), but the inner business logic remains untouched



The Dependency Rule

Presentation

Composables, ViewModels - what the user sees and interacts with

depends on ↓

Domain

Business rules, use cases, models - the core of the app, no framework imports

depends on ↓

Data

Repositories, API clients, Room DAOs - where data actually comes from

Dependencies point downward only. Domain never imports from Presentation or Data.



Mapping Layers onto a Real Android Project



presentation/

AccountListScreen.kt, AccountsViewModel.kt



domain/

Account.kt, AccountsRepository.kt (interface only), GetAccountsUseCase.kt



data/

AccountsRepositoryImpl.kt, AccountsApiService.kt, AccountDao.kt

Notice: the domain layer defines AccountsRepository as an interface. The data layer provides AccountsRepositoryImpl. This inversion is what makes the dependency point inward - Domain doesn't know Data exists.



Block 2 - Clean Architecture & Layering

Skill Check ✓

- State the Dependency Rule in one sentence.
- Which layer is allowed to import Jetpack Compose, and which layer must never import it?
- Why does the Domain layer define AccountsRepository as an interface rather than a concrete class?



Use Cases & Domain Layer

- What a Use Case Actually Is
- One Job, One Class
- Where Business Rules Actually Live



What is a Use Case?

- A **use case** is a single class that contains application-specific business logic for one specific task or user action
- It acts as the bridge between your outer layers (like the UI or web controllers) and your core business rules and entities



Use Case Key Responsibilities

- **Orchestration:** Coordinates the flow of data to and from domain entities and calls repositories to fetch or save data
- **Single Responsibility:** Handles one specific operation the system can perform (e.g., LoginUserUseCase, SendPaymentUseCase)
- **Isolation:** Knows *what* the application needs to do, but does not know *how* outer details like databases, frameworks, or user interfaces work



Use Case Characteristics

- **Inward Dependencies:** It sits in the application/domain layer and depends only on abstractions (interfaces), allowing outer technologies to change without affecting the core logic
- **Synonyms:** Often referred to as an **interactor**
- **Testability:** Because it has no external dependencies on UI frameworks or databases, it can be tested quickly via pure unit tests



Use Case Examples

```
class GetAccountsUseCase(private val repository: AccountsRepository) {  
    suspend operator fun invoke(): List<Account> {  
        return repository.fetchAccounts()  
    }  
}
```

```
class TransferFundsUseCase(  
    private val repository: AccountsRepository,  
    private val eligibilityService: TransferEligibilityService  
) {  
    suspend operator fun invoke(amount: Double, from: Account, to: Account): Result<Unit> {  
        val eligible = eligibilityService.canTransfer(amount, from)  
        if (!eligible) return Result.failure(InsufficientFundsException())  
        repository.transfer(amount, from, to)  
        return Result.success(Unit)  
    }  
}
```



One Job, One Class



Independently testable

A use case's test only needs to fake exactly the dependencies it declares - nothing more



Independently reusable

GetAccountsUseCase can be called from a ViewModel, a widget, or a background sync job identically



Self-documenting

The class list of a domain/ package IS a list of everything the app can do, in plain English



Block 3 - Use Cases & Domain Layer

Skill Check ✓

- What does the operator `invoke()` convention let you do when calling a use case?
- How is a use case more granular than Module 7's Service layer?
- Name one concrete benefit of one-job-one-class use cases beyond just organization.



The Repository Pattern

- Abstracting Data Sources
- Mapping DTOs
- The Full Chain
- Repository vs. Data Source



The Repository Pattern

- The **Repository Pattern** is a design pattern that abstracts data access logic
- It acts as a mediator between your data sources and the rest of your application
 - Sources could be local database, a network API, or cache
- It provides a clean API for the UI layer (via the ViewModel) to request data without needing to know *where* the data comes from or *how* it is fetched
- The data access logic could be as simple as accessing a network API
- Or it could provide local cache first with fallback to async API refreshing the local cache
- The point is that the UI layer is unaware of these details
 - And the details could be changed at any time without impacting the UI



Repository Lives in Which Layer?

- Repositories are pulled in two different directions:
- ViewModels (from the Presentation layer) need to interact with repositories
 - This implies that repositories need to be in the Presentation or the Domain layer
- Repositories need to interact with Data Sources (from the data layer)
 - This implies that repositories need to be in the data layer
 - The domain layer does NOT depend upon the data layer
- The usual solution to this dilemma is to have a repository interface in the domain layer
 - The Presentation layer can depend upon this interface
 - If you have UseCases (domain layer) they can also depend upon this interface
- And have the implementations of that interface in the data layer
 - These implementations depend upon the domain layer



Putting It Together: the Full Chain



The Composable only knows the ViewModel. The ViewModel only knows a Use Case. The Use Case only knows a Repository interface. Nothing upstream knows how data is actually fetched - that knowledge stays entirely inside the Repository's concrete implementation, injected by Hilt.



Repository Interface

- Standard Kotlin interfaces are usually used to describe the functionality provided by a repository

Code Bite

```
interface AccountRepository {  
    suspend fun getAccounts(): List<Account>  
    suspend fun getAccount(id: Int): Account?  
}
```



Repository Implementation

- More than one class could implement the repository interface
- This allows for a range of behaviors
 - Simply query an API and return the data
 - Return cached data if exists, otherwise query the API and cache and return the results
 - Return the cached data immediately then refresh the cache with remote data
 - Possibly multi-level caching (memory and local data)

Code Bite

```
class CachingAccountRepository(  
    private val apiSource: AccountDataSource,  
    private val memSource: AccountDataSource  
) : AccountRepository {  
  
    override suspend fun getAccounts():  
        List<Account> {  
        // logic to return cached data, if exists  
        // otherwise query the api and return it  
    }  
}
```



Data Sources

- It is often beneficial to repeat this pattern (an interface with potentially multiple implementations) for one more layer
- Separate data source classes could isolate the code for querying a remote API from the code that queries a local data store or a local memory cache
- The repository can then act as a high-level mediator that manages data operations using business logic
- The data source is a low-level component responsible for direct physical connection to a specific storage mechanism



Mapping DTOs to Domain Models

- The domain and presentation layers use what is called a domain model
 - i.e. the set of attributes a given entity has
- The set of attributes a data source receives from its storage mechanism may not match the domain model
 - Translation is therefore needed
- Best practices recommend a separate class to do the translation
- Often, companion objects are used to make invocation simple
 - There is no need to create an instance of the mapper

Code Bite

```
class AccountMapper {  
    companion object {  
  
        fun toDomain(dto: AccountDto): Account {  
            return Account( . . . )  
        }  
  
        fun toDto(domain: Account): AccountDto {  
            return AccountDto( . . . )  
        }  
    }  
}
```



Repository vs. Data Source

Data Source	Repository
Communicates with one source	Coordinates application data
Understands how data is stored or retrieved	Presents data in terms the domain understands
May return API DTOs or database entities	Returns domain models
May be local, remote, or in-memory	Decides which source to use
Performs storage-specific operations	May enforce caching and freshness policies



Block 4 - The Repository Pattern

Skill Check ✓

- In the full chain, what does the ViewModel directly depend on?
- Why can the Repository's data source change from network to local database without the ViewModel changing at all?
- What role does Hilt play in assembling this whole chain at runtime?



Dependency Injection with Hilt

- Manual DI, Revisited
- Why a Framework Now?
- Hilt Annotations
- Injecting into ViewModels



Manual DI, Revisited

- Manual DI doesn't stop working at scale
- It just becomes a LOT of repetitive, error-prone wiring code, spread across every screen, that someone has to maintain by hand.

Code Bite

```
class AccountsViewModel(  
    private val repo: AccountsRepository  
) : ViewModel() { . . . }  
  
// But who constructs the AccountsViewModel,  
// and who constructs the  
// AccountsRepositoryImpl it needs, and the  
// AccountsApiService THAT needs,  
// and does so correctly across dozens of  
// Activities, Fragments, and ViewModels,  
// respecting Android's own lifecycle  
// at every step?
```



Why a Framework?



Lifecycle-aware scoping

Hilt automatically ties a dependency's lifetime to an Activity, Fragment, or the whole app - manual DI has no concept of this



Less repetitive wiring

One `@Inject` constructor replaces hand-written factory code repeated across every screen that needs the same dependency



Compile-time verified

Hilt catches a missing or misconfigured dependency at compile time, not as a runtime crash



Installing Plugins for Hilt

- There are two plugins required:
- Hilt plugin
 - Automatically re-writes your compiled Android classes so they extend the correct Hilt-generated base classes for DI to work
 - Helps integrate with Dagger to validate dependency graphs and ensure there are no circular dependencies
- KSP plugin
 - Reads custom annotations and uses them to trigger automated code generation

Code Bite

```
// in project-level build.gradle
plugins {
    id("com.google.dagger.hilt.android")
    version "2.51.1" apply false
    id("com.google.devtools.ksp")
    version "2.0.21-1.0.25" apply false
}

// in app-level build.gradle
plugins {
    id("com.google.dagger.hilt.android")
    id("com.google.devtools.ksp")
}
```



Installing Dependencies for Hilt

- There are three dependencies needed
 - hilt-android and hilt-compiler are core dependencies
 - hilt-compiler is only needed at build time, so it is included using ksp() instead of implementation()
 - hilt-navigation-compose integrates Hilt into Jetpack Compose and provides custom annotations like @hiltViewModel()

Code Bite

```
// in app-level build.gradle
dependencies {
    . . .
    implementation("com.google.dagger:hilt-
android:2.60.1"
    ksp("com.google.dagger:hilt-
compiler:2.51.1")

    implementation("androidx.hilt:hilt-
navigation-compose:1.4.0")

}
```



Initializing Hilt

- Hilt needs a class that inherits from Application and is decorated with `@HiltAndroidApp`
- This class does not need to do anything else
 - It does need to be registered in the AndroidManifest.xml file using the `android:name` attribute
- It triggers Hilt's code generation during compilation and provides a base application class to contain the auto-generated code

Code Bite

```
@HiltAndroidApp  
class MainApplication: Application()
```

```
// in AndroidManifest.xml  
<application  
    android:name=".MainApplication"  
    . . .
```



Marking the Application Entry Point

- The main activity that comprises your application's entry point needs to be annotated with `@AndroidEntryPoint`
 - This allows Hilt to inject dependencies into it

Code Bite

```
@AndroidEntryPoint
class MainActivity: ComponentActivity() {
    . . .
}
```



Injecting ViewModels

- ViewModel classes need to be annotated with `@HiltViewModel`
- When they are needed in a composable, use the `hiltViewModel()` function to resolve them instead of the built-in `viewModel()` function

Code Bite

```
@HiltViewModel
class AccountListViewModel(): ViewModel() {
    . . .
}

@Composable
fun AccountList(
    viewModel: AccountListViewModel =
        hiltViewModel()
) {
    . . .
}
```



@Inject

- Concrete classes that Hilt needs to be able to inject should use the @Inject annotation on their constructor
 - Even if the class has no constructor parameters of its own
- If the class does have constructor parameters, Hilt can inject them
 - Provided their constructors are annotated with the @Inject decorator

Code Bite

```
class InMemoryAccountDataSource
    @Inject constructor(): AccountDataSource {
        . . .
    }

class DefaultAccountRepository
    @inject constructor(
        private val dataSource:
                                InMemoryAccountDataSource
    ): AccountRepository {
        . . .
    }
```



Interfaces and @Binds

- What if the @Inject-annotated constructor has parameters that are an interface type?
- Hilt needs to be configured to know which specific implementation should be used for a given interface
- This is done in a module
 - An abstract class that is annotated with @Module
- It contains functions that map an interface to a specific implementation
 - This is done using the @Binds annotation

Code Bite

```
@Module
@InstallIn(SingletonComponent::class)
abstract class AppInjectionModule {

    @Binds
    @Singleton
    abstract fun bindAccountDataSource(
        implementation: RemoteAccountSource
    ): AccountDataSource

}
```



Lifetime of Injected Instances

@InstallIn Component	Matching Scope	Created At
SingletonComponent::class	@Singleton	Application#onCreate()
ActivityRetainedComponent::class	@ActivityRetainedScoped	Activity first onCreate
ViewModelComponent::class	@ViewModelScoped	ViewModel created
ActivityComponent::class	@ActivityScoped	Activity#onCreate()
FragmentComponent::class	@FragmentScoped	Fragment#onAttach()
ViewComponent	@ViewScoped	View#super()
ServiceComponent	@ServiceScoped	Service#onCreate()



Hilt Annotations & Injecting into a ViewModel

@HiltViewModel

```
class AccountsViewModel @Inject constructor(  
    private val repository: AccountsRepository  
) : ViewModel() {  
    // identical body to the manual-DI version - only the  
    // construction mechanism changed  
}
```

@Module

@InstallIn(SingletonComponent::class)

```
object RepositoryModule {  
    @Provides  
    fun provideAccountsRepository(api: AccountsApiService): AccountsRepository =  
        AccountsRepositoryImpl(api)
```



@Provides

- Sometimes you cannot use standard constructor injection
 - You do not own the class (it is from a 3rd party library)
 - You need custom configuration logic
 - You want to pass specific arguments to the instance being injected
- You can create a provider function
 - Annotated with @Provides
 - Must be inside a Hilt module
 - By convention, name starts with "provides"
 - Its parameters will be injected automatically
 - Its return type defines what type is being provided

Code Bite

```
@Module
@InstallIn(SingletonComponent::class)
object AppModule {
    @Provides
    @Singleton
    fun provideOkHttpClient(): OkHttpClient {
        return OkHttpClient.Builder()
            .readTimeout(30, TimeUnit.SECONDS)
            .build()
    }
}
```



Block 5 - Dependency Injection with Hilt

Skill Check ✓

- What does manual DI stop being able to do gracefully at Android application scale?
- What's a concrete capability Hilt provides that hand-written constructor injection doesn't?
- What does `@HiltViewModel` plus an `@Inject` constructor let Hilt do automatically?



StateFlow, LiveData & Clean Architecture Synthesis

- StateFlow vs. LiveData
- Which One to Reach For
- Tying the Whole Module Together



StateFlow vs. LiveData



StateFlow

Kotlin Coroutines-native, requires an initial value, the modern default for new code



LiveData

Android-lifecycle-native, older API, still extremely common in existing enterprise codebases



Recommendation

Use StateFlow for all new code in this bootcamp and going forward; recognize LiveData fluently for legacy work



StateFlow vs. Compose State (mutableStateOf)

Feature	StateFlow	mutableStateOf
Primary Location	ViewModel/Data Layer	UI Layer/Composables
Ecosystem	Part of pure Kotlin Coroutines	Bound tightly to Jetpack Compose
Capabilities	Supports powerful operators (map, combine, filter)	Simple value holder optimized for recomposition
Multiplatform?	Highly compatible with Kotlin Multiplatform	Limited to Compose-supported UI layers



Tying the Whole Module Together

MVVM separates concerns. Use cases give each business operation its own testable, reusable home. Hilt makes the pieces swappable without hand-wiring. The Repository gives the app one source of truth. StateFlow makes state observable.

This is the same architectural arc Module 7 built for iOS - different keywords (Hilt instead of manual protocols, StateFlow instead of Combine), same engineering judgment underneath.



Review: the Full Chain



The Composable only knows the ViewModel. The ViewModel only knows a Use Case. The Use Case only knows a Repository interface. Nothing upstream knows how data is actually fetched - that knowledge stays entirely inside the Repository's concrete implementation, injected by Hilt.



Block 6 - StateFlow, LiveData & Synthesis

Skill Check ✓

- Which should you reach for in new code, StateFlow or LiveData - and why does the other still matter?
- Name the five pieces of this module's architectural chain, in order, from screen to data source.
- What Module 7 concept does Hilt correspond to, adapted for Android's specific lifecycle needs?



Refactor: TransferActivity

- Starter file contains a Massive Activity mixing networking, validation, and UI.
- Extract a TransferViewModel with zero Android framework imports beyond androidx.lifecycle.
- Extract a TransferFundsUseCase encapsulating the eligibility check and the transfer call.
- Inject the Repository and UseCase via Hilt - no manual construction, no singletons.
- The refactored ViewModel must be unit-testable using a fake repository, with no real network or Hilt container required.

Module 13 Complete

You can now recognize a Massive Activity and extract an MVVM structure from it, reason about Clean Architecture's dependency rule on Android, extract focused use cases, inject dependencies with Hilt instead of wiring by hand, split data access into a Repository, and choose correctly between StateFlow and LiveData. Module 14 builds the real networking and persistence layer this architecture was designed to hold.

Next: Module 14 - Networking & Persistence